

DEPENDENCY SECURITY BEFORE EXECUTION

WARDEN

**Every dependency change becomes
a verified transaction.**

Plan the complete graph. Approve only the authority required. Install
with scripts suppressed. Prove what landed.

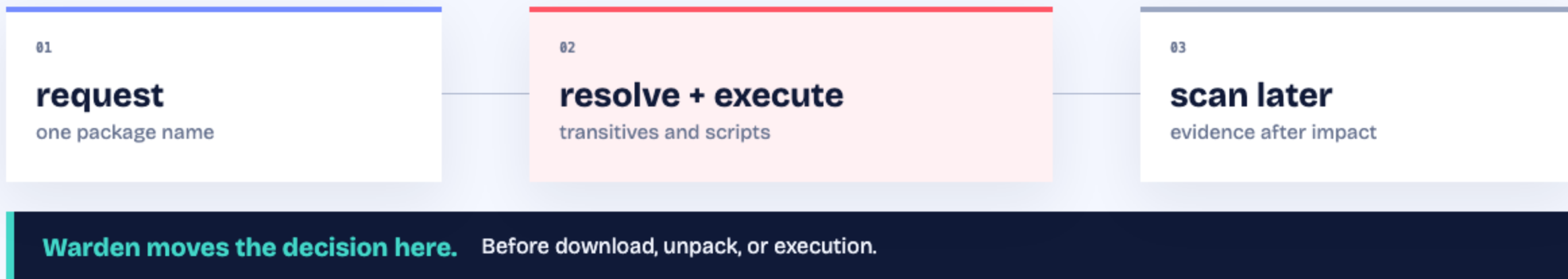


```
$ npm install esbuild █
```

THE PROBLEM

The dangerous moment happens before a scanner can report it.

A lifecycle script runs with your permissions. By the time a post-install scan starts, credentials or source may already be gone.



THE HIDDEN CHANGE

You type one package. The manager installs a graph.



**27 packages
change**

Checking only the name you typed leaves 26 transitive packages unexamined.

Four transaction stages close four different failure modes.

01

PLAN

See the complete prospective graph and analyze every change.

prevents blind transitives

No complete analysis means no confident **ALLOW**.

02

APPROVE

Name the exact script or coverage exception that needs authority.

prevents broad bypasses

03

APPLY

Replay the planned request with package scripts suppressed.

prevents pre-review execution

04

VERIFY

Compare the observed graph and project checks with the plan.

prevents silent drift

What will this command actually change?

● ● ● prospective graph

»» SKIP

● LIVE

```
$ warden plan -- npm install esbuild
✓ resolving the prospective dependency graph · 27 6.1s
✓ vetting changed packages · 27/27 19.6s
Warden plan: npm install esbuild
+ 26 transitive packages · 27 of 27 analyzed
NEEDS_APPROVAL esbuild@0.28.1 has a postinstall script
approval binds version + integrity + hook + script body
script policy: suppressed
$ warden plan -- npm install esbuild
```

GRAPH

27 changed

direct and transitive

COVERAGE

27 analyzed

nothing silently skipped

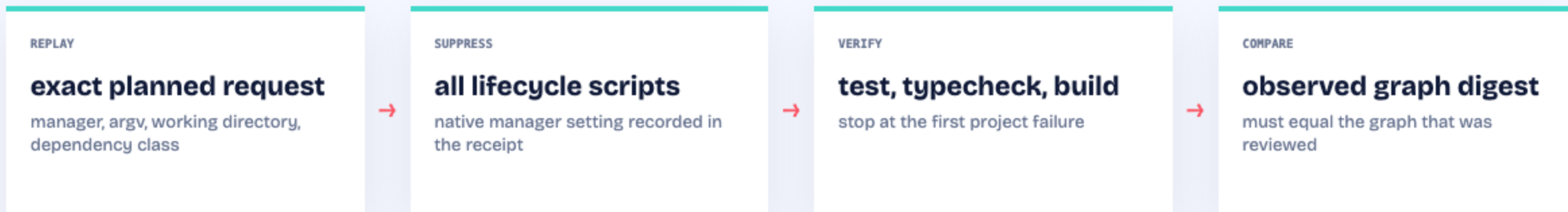
AUTHORITY

1 requirement

the exact postinstall hook

The approval is bound to package, version, tarball integrity, hook, and script body. The script still does not run.

Apply detects drift, verifies the project, and leaves a receipt.



TRANSACTION RECEIPT

result: applied

graph reviewed = graph observed
script policy = suppressed
artifact coverage = complete

WHEN ANY CHECK FAILS

Restore the manifest and every lockfile.

Warden does not claim to restore node_modules or side effects from verification.

The receipt is the control that survives a local bypass.

GUIDANCE	Teach the safe loop	Instructions and agent setup make the intended path obvious.	can be ignored
INTERCEPTION	Mediate package-manager commands	Shims cover npm, pnpm, Yarn, Bun, npx, bunx, exec, dlx, ci, rebuilds, and global installs.	absolute paths and containers can bypass it
VERIFICATION	Refuse the pull request without a valid receipt	<code>warden ci --require-transaction-receipt</code>	does not trust the developer machine

dependency graph changed →

receipt verifies →

MERGE

Warden checks the release delta and the repository surfaces around it.

PUBLISHED PACKAGE

**resolve → integrity → release diff → AST
scan → name intel → deterministic score**

Finds known malware, name attacks, new execution, provenance changes, credential access, egress, destructive calls, and obfuscation combined with capability.

`warden check` is deterministic. Verdicts are cached by tarball integrity and analyzer version, so repeat checks stay fast without reusing stale analysis.

LOCKFILE

Where will bytes come from?

Off-registry hosts, plaintext transport, weak integrity, git and file sources.

SCRIPTS

What code runs during install?

Lifecycle hooks across the installed graph, including pipe-to-shell and credential access.

CONFIG

Where will credentials be sent?

Lookalike registries, plaintext tokens, and disabled TLS without echoing secret values.

Native safety settings should not depend on remembering four dialects.

REPOSITORY INTENT

approved scripts
minimum release age
block exotic sources
reverify lockfiles
block downgrades



npm

ignore-scripts, min-release-age, source controls

pnpm

strict builds, minimum age, exotic-subdependency controls

Yarn

script control, hardened mode, age gate

Bun

install script suppression

HONEST GAP REPORTING

If the manager cannot express a rule, Warden says so and enforces it in the plan and receipt gate.

The invoked manager, packageManager field, lockfile, project config, and PATH decide which tool Warden preserves.

A vulnerability fix is not safe until the candidate release passes the gate.

01 Find advisories

Audit direct dependencies through OSV.

02 Build repair plans

Compare minimal and latest candidates.

03 Gate every fix

Reject the official fix if its release is risky.

04 Verify in isolation

Install without scripts, run project checks, then apply.

● ● ● repair plan

▶▶ SKIP

● LIVE

```
$ warden doctor
auditing 3 direct dependencies against OSV advisories
critical acme-http@1.0.0 fixed in 1.0.1
BLOCK acme-http@1.0.1 new script + exfiltration + provenance downgrade
UNFIXABLE the official fix fails the supply-chain gate
verified acme-json@2.1.4 install ok · test ok · applied
1 of 2 issues fixed · exit 10
$ warden doctor
auditing 3 direct dependencies against OSV advisories
critical acme-http@1.0.0 fixed in 1.0.1
```

Code can compile and still fail the request.

prompt versus diff

SKIP LIVE

```
$ warden intent check
prompt "add rate limiting, keep retries, log every limited request"
3 claims extracted · 2 files changed
✓ delivered    add rate limiting
✓ preserved    retry logic untouched
x dropped      log every limited request
! scope creep  pagination.ts was never requested
! hallucinated axios.instance.throttle is not exported
BLOCK diff does not match the prompt · exit 20
$ warden intent check
prompt "add rate limiting, keep retries, log every limited request"
```

DROPPED

A requested behavior never appeared.

SCOPE CREEP

Unrequested files or behavior changed.

HALLUCINATED API

Added code calls an export that does not exist.

When `.warden/prompt.txt` exists, CI runs the same intent check automatically for changed JavaScript and TypeScript.

BUILT FOR AUTOMATION

An agent receives a decision, evidence, a fix, and a finish line.



Stable contract

JSON output, published schemas, typed errors, and exit codes 0, 10, 20, 30.

Safe data boundary

Registry-authored text is sanitized and quarantined as untrusted input.

Capability-aware setup

Adapter setup reports which guidance, interception, post-change, and tool layers are actually available.

Restricted tools

The generated tool manifest exposes read-only decisions and excludes project-changing commands.

`warden handoff` writes the failing finding, evidence, concrete fix, and `warden ci --reporter agent` finish line

Every command answers an operational question.

Can I trust this package or release?

check vet the artifact

explain show what changed and what to do

history trace publisher, provenance, and scripts

compare rank candidates on evidence

baseline record a trusted reference

scripts show hooks still awaiting approval

Are the guardrails actually active?

coverage publish the intercepted command matrix

integrations doctor inspect PATH, shims, adapters, and CI

detect classify manager, topology, framework, and tooling

init add config, workflow, hooks, and context

config choose protect or observe behavior

log render recorded verdict history

Can teams and systems depend on it?

install preserve the project's chosen manager

ci emit summary, JSON, annotations, agent, or SARIF

schema publish versioned report contracts

completions generate bash, zsh, and fish support

benchmark reproduce regression evidence

uninstall remove shims, config, cache, and shell setup

MEASURED AND BOUNDED

The evidence is reproducible. The limits are explicit.

12/12

curated attack shapes
stopped

0/9

benign shapes stopped

METHOD

Every case runs through the real resolver and transaction decision with complete graph coverage.

Regression corpus, not field accuracy.

Not a sandbox

Absolute paths and containers can bypass local shims.

Technical alpha: a working control plane with honest boundaries.

Scripts stay suppressed

Packages that require install hooks may fail verification.

Rollback is scoped

Manifest and lockfiles return. node_modules does not.

Receipts are unsigned

Evidence, not independent attestation.

THE OUTCOME

Keep the workflow. Add a chain of custody.



ONE DECISION CONTRACT

ALLOW • WARN • NEEDS_APPROVAL • BLOCK

local, package managers, automation, and CI

Sources

01 Sonatype software supply-chain report

03 OSV advisory database

05 Transaction model and limitations

02 USENIX package hallucination study

04 Warden benchmark corpus and method

06 Command interception coverage

Product claims correspond to the shipped command registry and repository documentation on this revision.